Clustered vs Non-Clustered Indexes: Choosing the Right One

Clustered. A clustered index stores the table's rows themselves in key order, so its leaf level is the data. Because rows can be sorted only one way, a table gets a single clustered index. Choose a narrow, unique, stable, steadily increasing key such as `staff_id`; range scans and ORDER BY on that key then read neighbouring pages sequentially.

Non-clustered. A non-clustered index is a separate sorted structure holding the search key, here `full_name`, plus a row locator pointing to where each row lives. Its order deliberately differs from the data order, so each match costs an extra hop to the row. A table can have many. Use them for columns that filter, join or sort but are not the main access path, and add INCLUDE columns so common queries never leave the index.

> **Key rule:** a clustered index requires the physical data order to match the index order.

Trade-off. Every index speeds reads but slows INSERT, UPDATE and DELETE and consumes storage. Index only what real queries need.

Security. The view `vw_StaffDirectory` exposes only staff_id and full_name, keeping salary and phone hidden.